AS

AeroSense

Embedded Aerospace Instrumentation and Telemetry Prototype

Sabrina Bryce

October 30, 2026

Executive Summary

AeroSense is an embedded instrumentation and telemetry prototype developed to explore sensor integration and real-time visualization in an aerospace-inspired application. Built around a Raspberry Pi Pico W, the system acquires motion measurements from an MPU6050 inertial measurement unit (IMU) and environmental measurements from a thermistor and photoresistor. MicroPython manages sensor acquisition, data processing, system control, and wireless communication.

A browser-based dashboard presents sensor measurements and system status through a live telemetry interface. Physical buttons and LED indicators provide user interaction and status feedback, while simulated flight functions demonstrate system arming and event-marker recording.

Development involved integrating multiple sensor interfaces, troubleshooting hardware connections, calibrating sensor readings, and implementing communication between the embedded system and browser interface. The completed prototype demonstrates the integration of low-cost sensing hardware, embedded programming, wireless communication, and interactive telemetry visualization. Future work should improve measurement accuracy through reference-based calibration, incorporate sensor fusion for orientation estimation, implement persistent data logging, and expand testing under controlled conditions.

Table of Contents

1.0 Project Motivation and Objectives 1

1.1 Project Overview 1

1.2 Project Requirements 1

1.3 Project Constraints 2

2.0 System Design 2

2.1 Hardware Architecture 2

2.2 Component Selection 4

3.0 Software and Telemetry Interface 4

3.1 Embedded Firmware 5

3.2 Wireless Communication 5

3.3 Dashboard and Visualization 6

3.4 Software Organization 6

4.0 Testing and Results 9

4.1 Hardware Testing 10

4.2 MPU6050 Testing 11

4.3 Thermistor Testing 11

4.4 Photoresistor Testing 12

4.5 Integrated System Testing 13

5.0 Discussion 13

5.1 Limitations 13

5.2 Future Work 14

5.3 Project Outcomes 14

References .....

List of Figures

Figure 1. AeroSense System Architecture 2
Figure 2. AeroSense Prototype Hardware 3
Figure 3. AeroSense Wiring Diagram 4
Figure 4. AeroSense Software Organization 6
Figure 5. AeroSense Telemetry Dashboard 7
Figure 6. Flight Systems in Armed State 8
Figure 7. Telemetry Status in Armed State 8
Figure 8. Dynamic Artificial Horizon 9
Figure 9. Flight Event Logging Interface 9

List of Tables

Table 1. Functional Requirements 1
Table 2. Hardware Configuration 3
Table 3. Hardware Testing Results 10
Table 4. MPU6050 Acceleration Measurements 11
Table 5. MPU6050 Gyroscope Measurements 11
Table 6. Thermistor Test Measurements 12
Table 7. Photoresistor Test Measurements 12

1.0 Project Motivation and Objectives

Aerospace instrumentation is fundamental to flight monitoring: sensors acquire measurements of vehicle motion, environmental conditions, and system status, while onboard electronics process and communicate this information through telemetry systems [1]. Integrating these functions into a reliable platform requires coordinating multiple sensors, electronic interfaces, processing methods, and communication components. Understanding how these components interact is an important part of instrumentation system design.

AeroSense explores this challenge through the development of a low-cost embedded instrumentation prototype. The project investigates how motion and environmental measurements can be acquired, processed, and communicated through a central system with local status indicators and a browser-based telemetry interface.

1.1 Project Overview

The objective of AeroSense is to develop a low-cost embedded instrumentation prototype that integrates motion and environmental sensing, data processing, and wireless telemetry. The system collects measurements from an inertial measurement unit (IMU), a thermistor, and a photoresistor, which are processed by a microcontroller and presented through a browser-based dashboard. Physical buttons and LED indicators provide local user interaction and status feedback, while simulated flight functions demonstrate system arming and event-marker recording.

1.2 Project Requirements

The prototype was designed around the functional requirements outlined in Table 1.

Table 1. Functional Requirements

Requirement	Description
Motion sensing	Acquire three-axis acceleration and angular velocity measurements.
Temperature sensing	Acquire ambient temperature measurements.
Light sensing	Acquire ambient light level measurements.
User inputs	Accept user inputs to control simulated system functions.
Status indication	Provide visual feedback about system states and events.
Data processing	Process sensor readings to display for monitoring.
Telemetry communication	Transmit sensor measurements and system status to a browser-based interface over Wi-Fi.
Data visualization	Display sensor measurements, orientation information, and system status through a dashboard.
Simulation functions	Support simulated system arming and flight-marker recording.
Low-cost implementation	Use readily available, low-cost components suitable for a benchtop prototype.

1.3 Project Constraints

The design and implementation of AeroSense were subject to the following constraints:

- **Hardware platform:** The Raspberry Pi Pico W's processing resources and available interfaces constrained the complexity of the embedded software and web interface.
- **Measurement accuracy:** The low-cost sensors support basic monitoring and demonstration but require appropriate calibration and validation before their measurements can be treated as quantitatively accurate.
- **Telemetry interface:** Hosting the web server directly on the Pico W limits the available processing resources and the complexity of the dashboard.
- **Prototype scope:** The breadboard-based system is intended for benchtop testing and demonstration. It is not designed or validated for use in an operational aircraft.

2.0 System Design

AeroSense uses a Raspberry Pi Pico W as the central controller, connecting the sensing, user input, status indication, and telemetry functions into a single embedded system. The overall system architecture is shown in Figure 1.

Figure 1. AeroSense System Architecture



The MPU6050 provides motion measurements, while a thermistor and photoresistor provide temperature and relative light-level measurements. The Pico W processes these inputs, monitors user interactions, and controls the LED indicators. Sensor measurements and system status are then made available to a browser-based dashboard through a Wi-Fi connection. This architecture integrates physical sensing, embedded processing, and data visualization within a compact, low-cost prototype.

2.1 Hardware Architecture

The hardware consists of a Raspberry Pi Pico W, an MPU6050 IMU, two analog sensing circuits, two pushbuttons, and three LEDs. The MPU6050 communicates with the microcontroller through the I^2C interface, while the thermistor and photoresistor connect to analog inputs through voltage-divider circuits. Pushbuttons provide digital inputs, and the LEDs provide visual status feedback. Table 2 summarizes the hardware configuration and GPIO assignments.

Table 2. Hardware Configuration

Component	Function	Connection
Raspberry Pi Pico W	Processing and communication	Main controller
MPU6050 IMU	Three-axis acceleration and angular velocity	SDA: GP4, SCL: GP5
Thermistor	Temperature measurement	ADC0 / GP26; voltage divider
Photoresistor	Relative light measurement	ADC1 / GP27; voltage divider
Green button	User input for simulated arming	GP21
Blue button	User input for flight-marker recording	GP20
Green LED	System status indication	GP18; series resistor
Red LED	Warning status indication	GP17; series resistor
Blue LED	Status and marker feedback	GP16; series resistor

The thermistor and photoresistor use 10 kΩ resistors in their voltage-divider circuits. The LEDs also use 10 kΩ series resistors for current limiting. Figure 2 shows the completed hardware prototype.

Figure 2. AeroSense Prototype Hardware

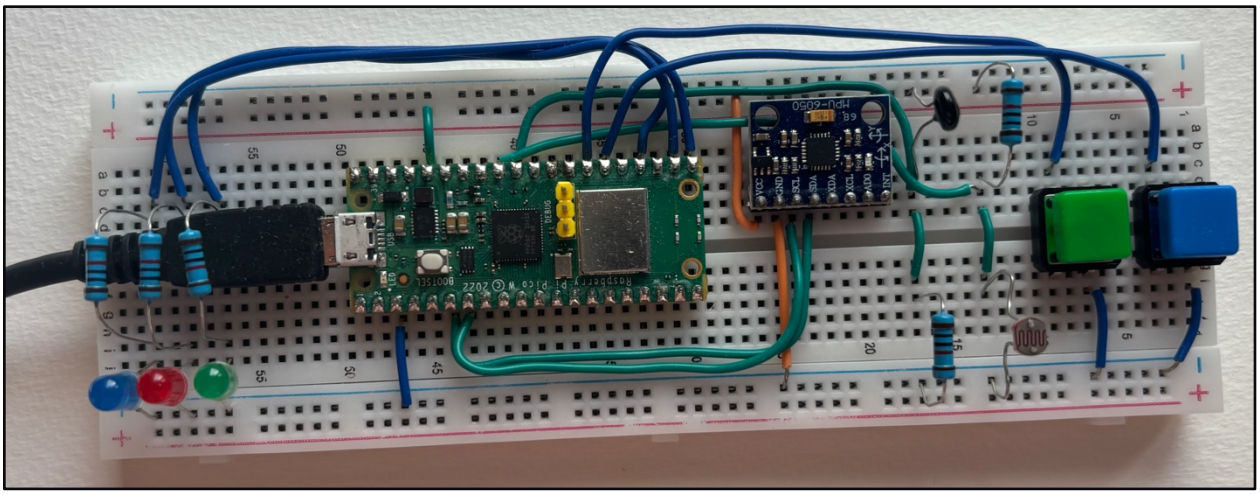
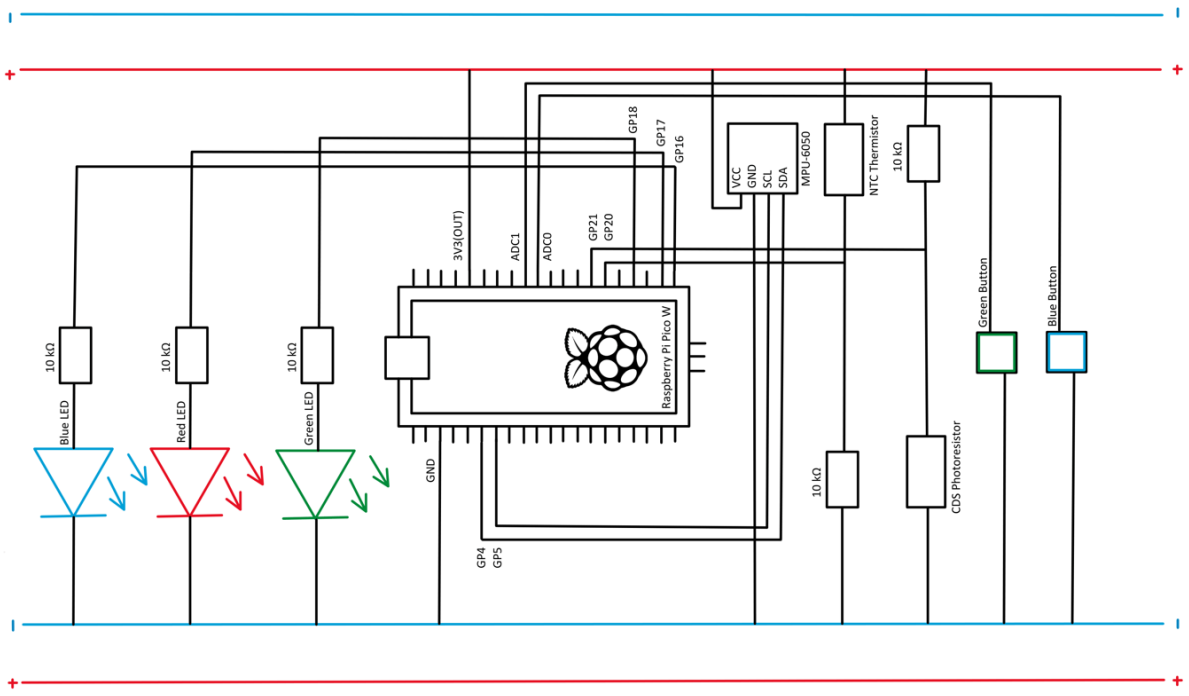


Figure 3 presents a simplified wiring diagram showing the connections between the hardware components.

Figure 3. AeroSense Wiring Diagram



2.2 Component Selection

The Raspberry Pi Pico W was selected as the central controller because it supports MicroPython, analog and digital I/O, I²C communication, and Wi-Fi connectivity [2]. These capabilities allowed sensor acquisition, system control, and telemetry hosting to be implemented on a single microcontroller.

The MPU6050 provides three-axis acceleration and angular velocity measurements through a digital I²C interface [3]. The thermistor and photoresistor provide temperature and relative light-level measurements through analog voltage-divider circuits using 10 kΩ resistors. Pushbuttons support user interaction, while LEDs provide immediate visual feedback.

Together, these components enable the integration of sensing, embedded processing, user interaction, and wireless telemetry using readily available hardware.

3.0 Software and Telemetry Interface

The AeroSense software was developed in MicroPython [4], [5] and JavaScript, with HTML and CSS used to implement the browser-based dashboard. The embedded firmware manages sensor acquisition, data processing, user inputs, and LED control. The Raspberry Pi Pico W hosts a web server that supplies the dashboard files and telemetry data to connected clients.

The software is organized around three main functions: embedded sensing and control, wireless telemetry communication, and browser-based data visualization. These functions allow the prototype to integrate physical measurements, user interaction, and real-time monitoring.

3.1 Embedded Firmware

The embedded firmware runs in MicroPython on the Raspberry Pi Pico W. It reads acceleration and angular velocity from the MPU6050 through the I²C interface and acquires temperature and light measurements through the analog inputs connected to the thermistor and photoresistor.

The thermistor measurements are converted into estimated temperatures using a negative temperature coefficient (NTC) thermistor model and a calibration offset. Photoresistor readings are averaged across multiple samples and mapped to a relative light-intensity percentage using defined voltage limits. These processing steps convert the raw analog measurements into values suitable for display.

The firmware also monitors two physical pushbuttons and controls three LEDs. The green button toggles the simulated flight system between armed and standby states, while the blue button records an event marker and briefly illuminates the blue LED. The red LED activates when the configured temperature or tilt warning threshold is exceeded. Button debounce logic reduces the likelihood of registering a single press multiple times.

Roll and pitch are estimated from accelerometer measurements using trigonometric calculations, with offsets applied to adjust the displayed reference position. These estimates drive the dashboard's artificial horizon. The current implementation does not combine accelerometer and gyroscope measurements through sensor fusion; orientation display is intended for demonstration rather than precision attitude measurement.

3.2 Wireless Communication

The Pico W connects to a Wi-Fi network and hosts a web server on port 8080. The server responds to browser requests for the dashboard's HTML, CSS, and JavaScript files, as well as requests to the /data endpoint.

When the browser requests /data, the firmware collects the current sensor measurements and system states, packages them into a JSON object, and returns them in an HTTP response. The JavaScript dashboard requests updated telemetry approximately once per second and uses the returned data to refresh sensor measurements, orientation graphics, warning indicators, and flight-system status.

The interface also detects unsuccessful telemetry requests and updates its connection-status indicators. This architecture allows the Pico W to serve the dashboard and telemetry directly without requiring a separate computer to process the sensor data.

3.3 Dashboard and Visualization

The dashboard was developed using HTML, CSS, and JavaScript to provide a browser-based interface for monitoring the prototype. Its layout groups environmental measurements, three-axis acceleration, angular velocity, orientation, flight-system status, and telemetry information into separate panels. A dark colour scheme, grid elements, and contrasting status indicators provide an aerospace-inspired visual design.

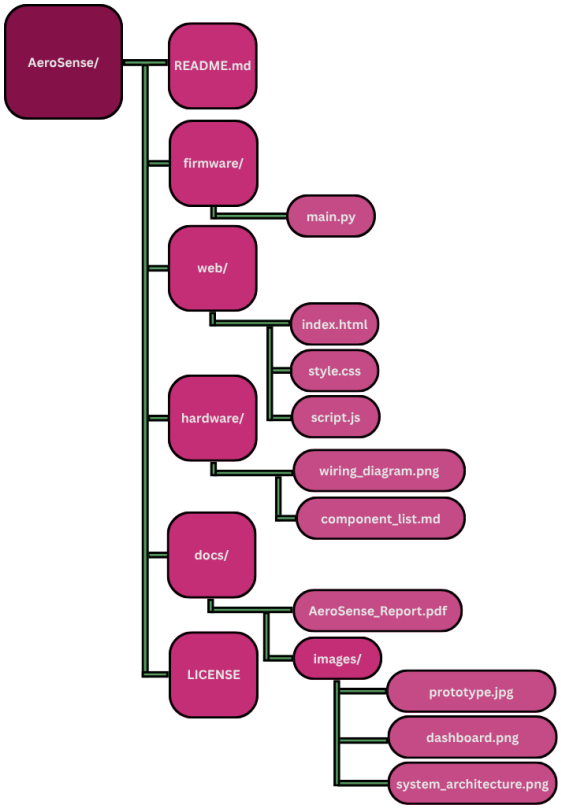
The artificial horizon responds to the calculated roll and pitch estimates. Smoothing is applied to successive orientation updates to reduce abrupt visual changes, while numerical readings display the corresponding estimated angles. The dashboard also presents the simulated flight-system state, event-marker count, and warning conditions. Temperature warnings activate above 35 °C, while tilt warnings activate when the absolute roll or pitch estimate exceeds 30°.

The mission event log records changes detected by the dashboard, including changes in arming state, new event markers, and warning activation or clearance. These entries are maintained in the browser session and are not saved to persistent storage. The event log therefore provides a session-level record of interface events rather than persistent flight-data recording.

3.4 Software Organization

Figure 4 illustrates the organization of the AeroSense software and the relationship between the embedded firmware and browser interface.

Figure 4. AeroSense Software Organization



Figures 5–9 show the dashboard layout, simulated flight states, telemetry status, artificial horizon, and event-logging interface.

Figure 5. AeroSense Telemetry Dashboard

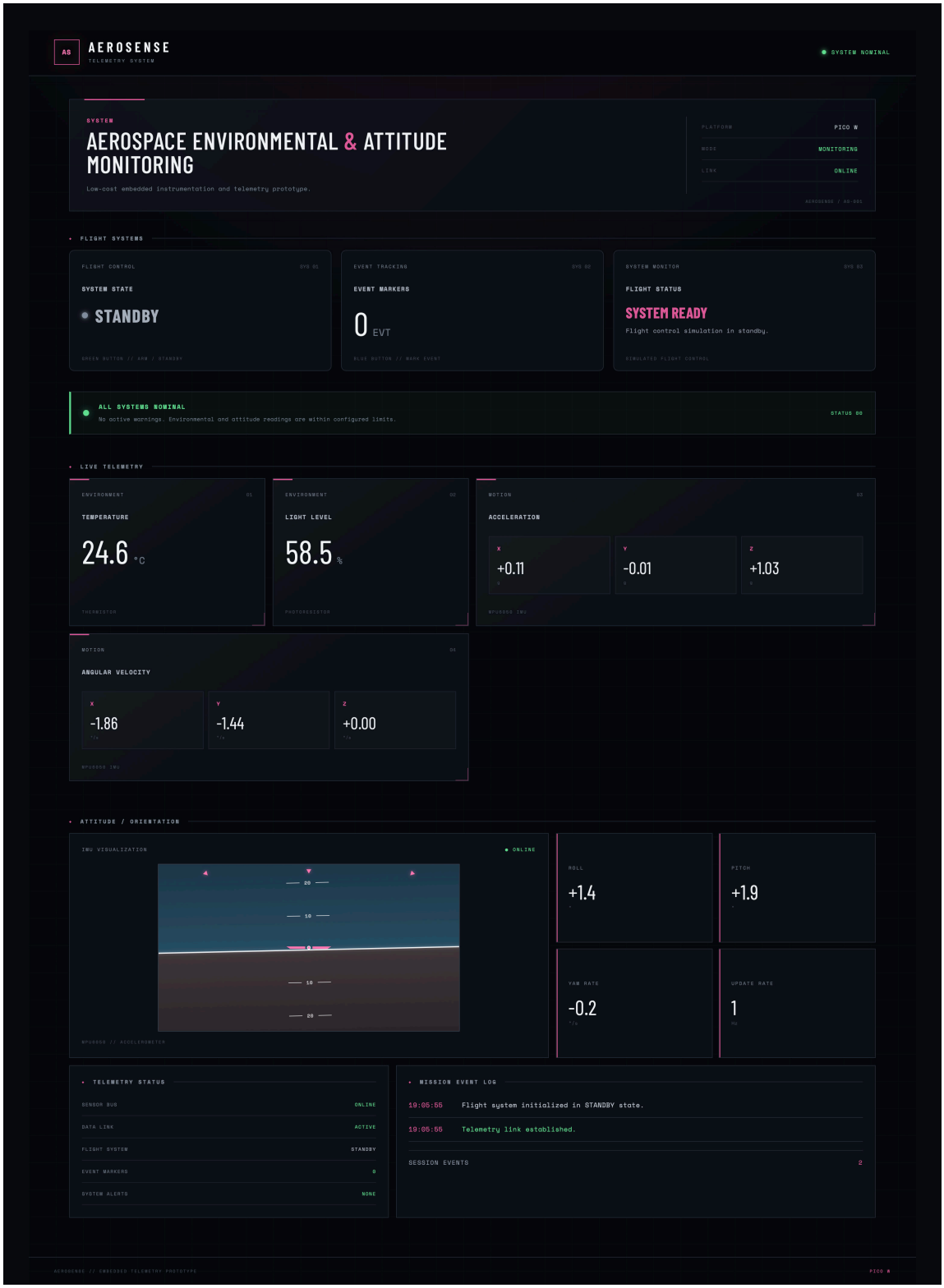


Figure 6. Flight Systems in Armed State

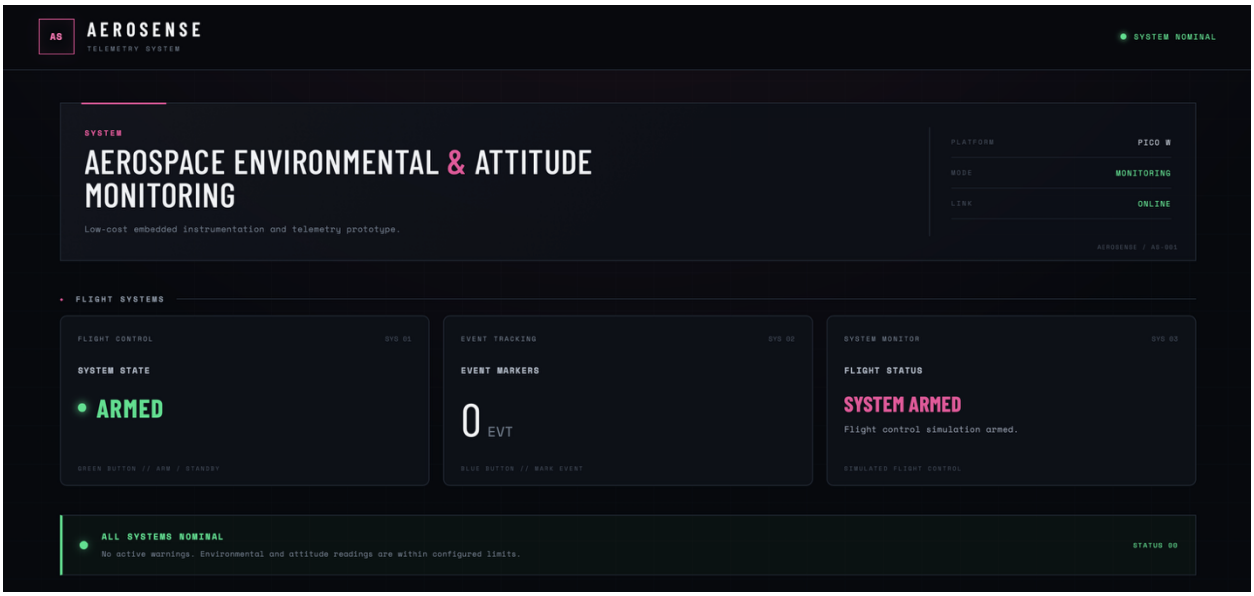


Figure 7. Telemetry Status in Armed State



Figure 8. Dynamic Artificial Horizon

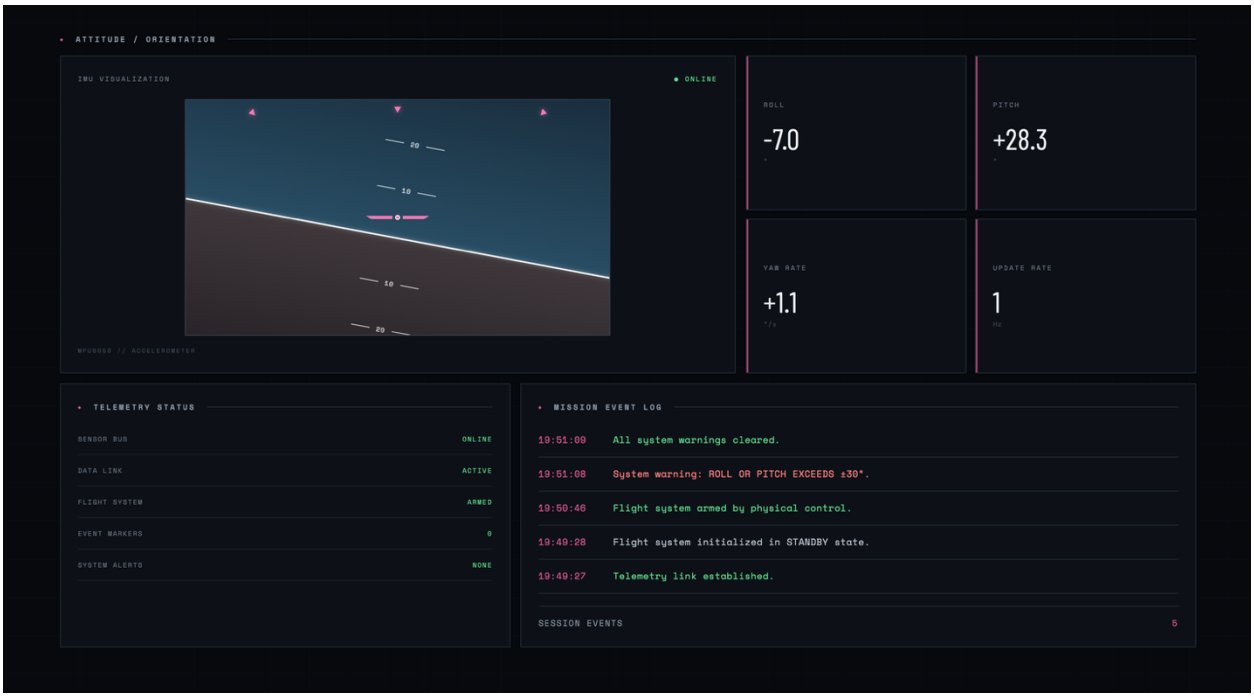
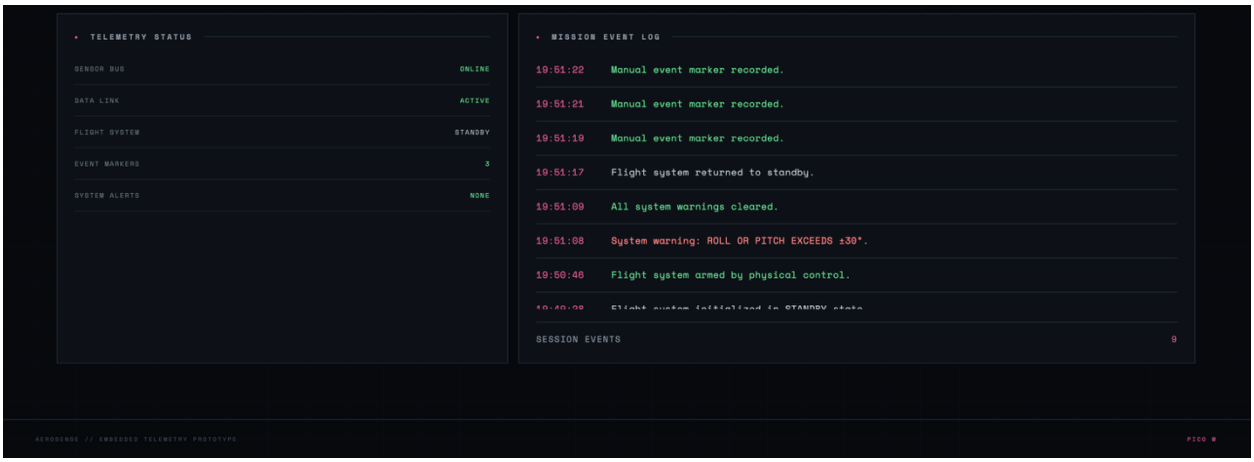


Figure 9. Flight Event Logging Interface



4.0 Testing and Results

AeroSense was tested incrementally to verify individual components before evaluating the integrated system. Hardware testing examined communication with the MPU6050, analog readings from the thermistor and photoresistor, digital inputs from the pushbuttons, and LED output control. The MPU6050 was also evaluated under stationary and moving conditions to assess its response to changes in motion.

Following component-level testing, the integrated system was evaluated through the browser-based dashboard. These tests examined telemetry updates, sensor visualization, simulated flight-system state changes, event-marker recording, and warning indicators.

Testing focused on confirming functional behaviour under benchtop conditions. Measurement accuracy was not established through comparison with calibrated reference instruments.

4.1 Hardware Testing

Table 3 summarizes the hardware tests and the observed behaviour of the individual components.

Table 3. Hardware Testing Results

Component or Function	Test Procedure	Observed Result
MPU6050 communication	Read acceleration and angular velocity through I^2C.	The sensor returned measurements that changed when the device was moved or rotated.
Photoresistor	Compare readings under different lighting conditions.	The calculated light percentage decreased in darkness and increased under brighter illumination.
Thermistor	Compare readings under different temperature conditions.	The temperature increased when warmed and decreased in cool conditions.
Green pushbutton	Press the button repeatedly while monitoring the system status.	Each press toggled the simulated flight system between armed and standby states.
Blue pushbutton	Press the button while monitoring the event-marker count.	Each registered press incremented the marker count and briefly activated the blue LED.
Blue LED	Register an event marker.	The LED illuminated briefly following a registered marker event.
Red LED	Evaluate the configured temperature and orientation warning conditions.	The LED activated when the temperature exceeded 35 °C or the absolute estimated roll or pitch exceeded 30°.
Green LED	Toggle the simulated flight system between states.	The LED indicated whether the system was armed.

4.2 MPU6050 Testing

The MPU6050 was evaluated under stationary and moving conditions to verify sensor communication and response to changes in motion. Table 4 presents the acceleration measurements and their total magnitudes, while Table 5 presents the gyroscope measurements and their total magnitudes.

Table 4. MPU6050 Acceleration Measurements

Test Condition	Acceleration X (g)	Acceleration Y (g)	Acceleration Z (g)	Total Magnitude (g)
Stationary, initial orientation	+0.10	-0.05	+0.99	1.00
Stationary, alternate orientation	+0.08	-0.09	+1.02	1.03
Rotated or moved	+0.15	-0.46	-0.76	0.90

Table 5. MPU6050 Gyroscope Measurements

Test Condition	Gyroscope X (°/s)	Gyroscope Y (°/s)	Gyroscope Z (°/s)	Total Magnitude (°/s)
Stationary, initial orientation	-0.21	-0.07	+0.08	0.24
Stationary, alternate orientation	+0.06	-0.11	+0.10	0.16
Rotated or moved	+1.01	-2.33	-0.66	2.62

The stationary acceleration magnitudes were approximately 1 g, consistent with the accelerometer measuring gravitational acceleration. During movement, the acceleration and angular velocity readings changed in response to the sensor's motion. Small nonzero gyroscope readings were observed under stationary conditions, consistent with residual zero-rate bias or sensor noise.

The accelerometer measurements are used to calculate roll and pitch estimates for the artificial horizon. Because the current implementation derives these estimates from accelerometer measurements alone, the orientation display is most appropriate for demonstrating static orientation and simple attitude changes. Dynamic acceleration can affect these estimates, and a sensor-fusion algorithm could improve orientation tracking during movement.

4.3 Thermistor Testing

The thermistor was evaluated by monitoring the displayed temperature under ambient conditions, after applying a handwarmer, and after applying a covered ice pack. Successful operation is indicated by a stable reading under relatively constant conditions, an increase in

the estimated temperature as the sensor warms, and a decrease as it cools. Table 6 presents the measured results during testing.

Table 6. Thermistor Test Measurements

Test Condition	Measured Temperature	Observed Response
Ambient conditions	22.3 °C	Readings remained stable.
Sensor warmed	36.2 °C	Increase observed.
Sensor cooled	8.1 °C	Decrease observed.

The firmware converts the analog voltage-divider measurement into an estimated temperature using an NTC thermistor model and a calibration offset. The direction of the temperature response provides a functional check of the sensing circuit and conversion algorithm. However, a plausible room-temperature reading does not establish measurement accuracy. Comparison with a reference thermometer would be required to evaluate the calibration.

4.4 Photoresistor Testing

The photoresistor was tested under different lighting conditions by covering the sensor, exposing it to normal room lighting, and directing a flashlight toward it. The firmware maps an input voltage of approximately 2.5 V to 0% relative light intensity and 0.2 V to 100%.

Table 7 presents the measured voltages and displayed response.

Table 7. Photoresistor Test Measurements

Test Condition	Approximate Voltage	Displayed Response
Sensor covered	2.5 V	Near 0% relative light intensity
Dim room	1.5 V	Approximately 43%
Normal room	1.0 V	Approximately 65%
Flashlight	0.15–0.25 V	Near 100%

The photoresistor responded to changes in illumination in the expected direction. The measured voltage decreased from approximately 2.5 V with the sensor covered to 0.15–0.25 V under a flashlight, while the calculated relative light intensity increased from approximately 0% to 100%. Intermediate readings under dim and normal room lighting produced corresponding intermediate percentages. These results demonstrate the functional response of the sensing circuit and the implemented voltage-to-percentage mapping. However, because the output was not calibrated against a reference light meter, the displayed percentage represents relative light intensity rather than absolute illuminance in lux.

4.5 Integrated System Testing

Integrated testing evaluated communication between the embedded firmware and browser dashboard, including sensor updates, orientation estimates, system status, warning indicators, and button inputs. The dashboard updated approximately once per second, while the simulated arming function, event-marker counter, and LED indicators demonstrated the intended control logic.

The artificial horizon responded to calculated roll and pitch estimates. Warnings were configured to activate above 35 °C or when the absolute estimated roll or pitch exceeded 30°. These thresholds are for demonstration and are not validated operational flight limits.

The prototype demonstrated integrated sensing, embedded processing, wireless communication, and browser-based visualization. Benchtop testing did not establish calibrated measurement accuracy, long-term reliability, or suitability for operational aerospace applications.

5.0 Discussion

AeroSense demonstrates the integration of low-cost sensors, embedded programming, wireless communication, and browser-based visualization within a single instrumentation prototype. Using the Raspberry Pi Pico W as the central controller allowed sensor acquisition, system control, and telemetry hosting to be implemented on one platform. The resulting dashboard provides a unified view of environmental measurements, motion data, orientation estimates, and simulated flight-system status.

A key development challenge was coordinating the hardware and software components while ensuring that sensor measurements were presented consistently through the interface. The project involved configuring I^2C and analog interfaces, troubleshooting sensor connections, applying calibration offsets, and developing communication between the embedded firmware and browser. Incremental testing helped isolate component-level issues before evaluating the integrated system.

5.1 Limitations

Several limitations remain. The thermistor and photoresistor rely on calibration parameters that have not been validated against reference instruments. The MPU6050 exhibits nonzero gyroscope readings at rest, and the current roll and pitch calculations use accelerometer data without sensor fusion. In addition, event markers are maintained in volatile memory, while the dashboard event log exists only in the browser session. The prototype therefore demonstrates telemetry and simulated flight-monitoring functions but does not provide persistent flight-data recording or validated aerospace-grade measurements.

5.2 Future Work

Future development could improve the system in four areas:

- Sensor calibration could be refined by comparing temperature and light measurements with reference instruments and characterizing gyroscope bias through repeated stationary measurements.
- A sensor-fusion algorithm could combine accelerometer and gyroscope data to improve orientation estimation during movement.
- Persistent data logging could record timestamped sensor measurements and system events for subsequent analysis.
- Additional testing could assess measurement repeatability, telemetry reliability, and system behaviour over extended operating periods.

5.3 Project Outcomes

AeroSense provides a practical foundation for exploring embedded instrumentation and telemetry. The project brings together hardware integration, sensor processing, software development, and interface design in a working benchtop prototype. It also identifies specific improvements needed to strengthen measurement confidence, expand monitoring capabilities, and support more rigorous future testing.

References

[1] IEEE, “Avionic Systems and Aerospace Instrumentation,” *IEEE Technology Navigator*. [Online]. Available: Avionic Systems And Aerospace Instrumentation | IEEE Technology Navigator. [Accessed: Oct. 29, 2026].

[2] Raspberry Pi Ltd., “Pico microcontroller boards,” *Raspberry Pi Documentation*. [Online]. Available: Raspberry Pi Pico Documentation. [Accessed: Oct. 29, 2026].

[3] TDK InvenSense, “MPU-6000 and MPU-6050 Product Specification,” document PS-MPU-6000A-00, rev. 3.4, Aug. 19, 2013. [Online]. Available: MPU-6000/MPU-6050 Datasheet.

[4] MicroPython, “class ADC – Analog to digital conversion,” *MicroPython Documentation*. [Online]. Available: MicroPython ADC Documentation. [Accessed: Oct. 29, 2026].

[5] MicroPython, “socket module,” *MicroPython Documentation*. [Online]. Available: MicroPython Socket Documentation. [Accessed: Oct. 29, 2026].